

EXPERIMENT 1: Cloud Storage Comparison

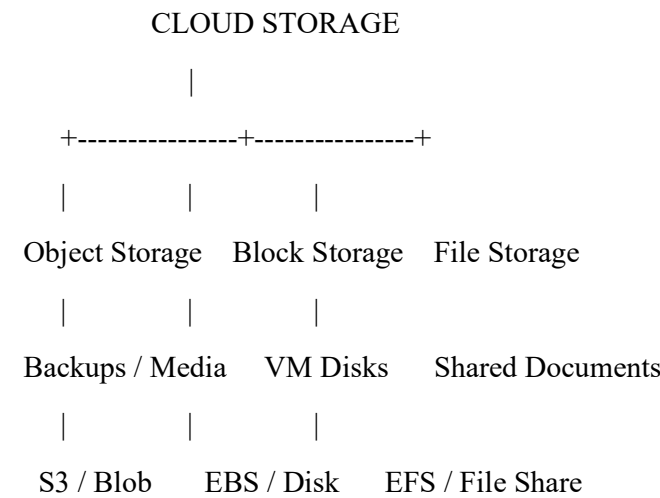
Title

Comparison of Object, Block, and File Storage Models for Different Cloud Workloads

Objective

To compare object, block, and file storage models and select the most suitable storage model for backups, VM disks, and shared enterprise documents.

Architecture Sketch



Storage Comparison

Feature	Object Storage	Block Storage	File Storage
Data unit	Objects	Blocks	Files
Access	API/HTTP	Attached disk	File system
Scalability	Very high	High	High
Best use	Backup/data archive	VM disks/databases	Shared documents

Procedure

1. Create or examine three cloud storage services: object, block, and file storage.
2. Upload a sample backup file to object storage.
3. Create a block storage volume and attach it to a virtual machine.
4. Create a file storage system and mount it to multiple instances.
5. Compare access method, scalability, performance, and sharing capability.
6. Map each storage model to an appropriate real-world application.
7. Record the observations in a comparison table.

Expected Output

Backup Data → Object Storage

Virtual Machine Disk → Block Storage

Shared Documents → File Storage

Analysis

Object storage is highly scalable and suitable for large backup files. Block storage provides low-latency disk-like access required by virtual machines. File storage provides a shared file system that can be accessed by multiple users or servers.

Conclusion

The experiment successfully compared the three major cloud storage models. Object storage is best for backups, block storage is best for VM disks, and file storage is best for shared enterprise documents.

EXPERIMENT 2: REST API Workflow for Cloud Student Portal

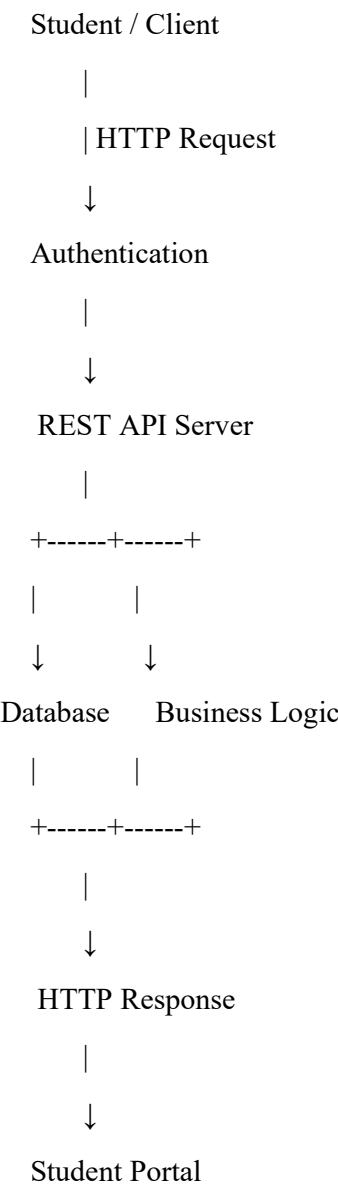
Title:

Development of a Simple REST API Workflow for a Cloud-Based Student Portal

Objective

To develop and demonstrate a REST API workflow showing client request, authentication, server processing, response, and cloud deployment.

Architecture Sketch:



Example API:

GET /api/students/101

Example Request:

GET /api/students/101

Authorization: Bearer <token>

Example Response:

```
{  
  "student_id": 101,  
  "name": "Rahul",  
  "course": "Computer Science",  
  "semester": 5  
}
```

Procedure:

1. Create a simple REST API using **Python Flask/FastAPI, Node.js, or another REST framework**.
2. Create a student resource containing student ID, name, course, and semester.
3. Implement a GET endpoint to retrieve student information.
4. Implement authentication using a token or API key.
5. Send an authenticated request using Postman or a browser/API client.
6. Validate the authentication token on the server.
7. Retrieve student information from the database.
8. Return the result as a JSON response.
9. Deploy the API on a cloud platform.
10. Test the deployed API using its public endpoint.

Expected Output:

Client



HTTP Request



Authentication



REST API



Database



JSON Response



Student Portal

Example:

Request:

GET /api/students/101

Response:

200 OK

```
{  
  "student_id": 101,  
  "name": "Rahul",  
  "course": "Computer Science"  
}
```

Analysis:

The REST API provides a standardized communication mechanism between the student portal and cloud backend. Authentication prevents unauthorized access, while JSON provides a lightweight data exchange format.

Conclusion:

The experiment successfully demonstrated a cloud-based REST API workflow involving request handling, authentication, database access, JSON response generation, and deployment.

EXPERIMENT 3: Serverless Image Processing

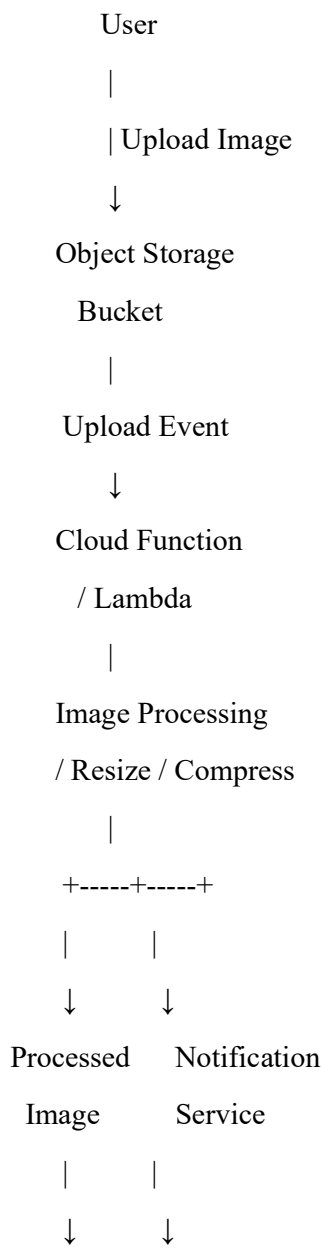
Title:

Design of a Serverless Event-Driven Image Processing System

Objective:

To design an event-driven serverless workflow using object storage, Function-as-a-Service (FaaS), and a notification service for automatic image processing.

Architecture Sketch:



Storage User Alert

Tools / Platforms:

Possible cloud services:

- Amazon S3 / Azure Blob / Google Cloud Storage
- AWS Lambda / Azure Functions / Google Cloud Functions
- Amazon SNS / similar notification service

Procedure:

1. Create an object storage bucket.
2. Configure the bucket to receive image files.
3. Create a serverless cloud function.
4. Configure the storage bucket to trigger the function when a new image is uploaded.
5. Implement image processing such as:
 - resizing,
 - compression,
 - thumbnail generation.
6. Store the processed image in another storage location.
7. Configure a notification service.
8. Send a notification after successful processing.
9. Upload a test image.
10. Observe the complete event-driven workflow.

Expected Output:

image.jpg



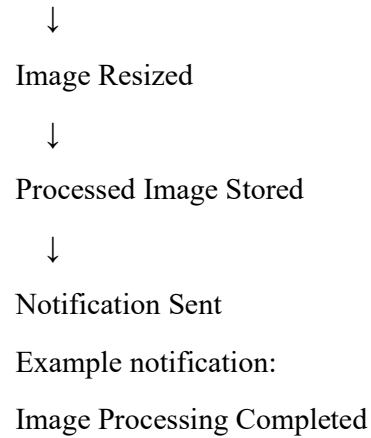
Object Storage



Upload Event



Cloud Function



File: image.jpg

Status: Successfully Processed

Output: thumbnail.jpg

Analysis:

The architecture is event-driven because the image upload automatically triggers the serverless function. No continuously running server is required. This improves scalability and reduces infrastructure management.

Conclusion:

The experiment successfully demonstrated a **serverless event-driven architecture** where an image upload automatically triggers processing and generates a notification.

EXPERIMENT 4: Edge Computing vs Fog Computing

Title

Comparison of Edge and Fog Computing for Smart Traffic Management

Objective

To compare **edge computing** and **fog computing** for a smart traffic management system based on architecture, latency, processing location, scalability, and deployment requirements.

Architecture – Edge Computing

Traffic Cameras

|

↓

Edge Device

|

Local Processing

|

↓

Traffic Signal

Processing takes place very close to the sensors/devices.

Architecture – Fog Computing

Traffic Cameras

|

↓

Local Gateway

|

↓

Fog Node

|

Processing

|

↓
Traffic Control Center

|
↓
Cloud

Processing is distributed across **local gateways, routers, and fog nodes** between devices and the cloud.

Comparison:

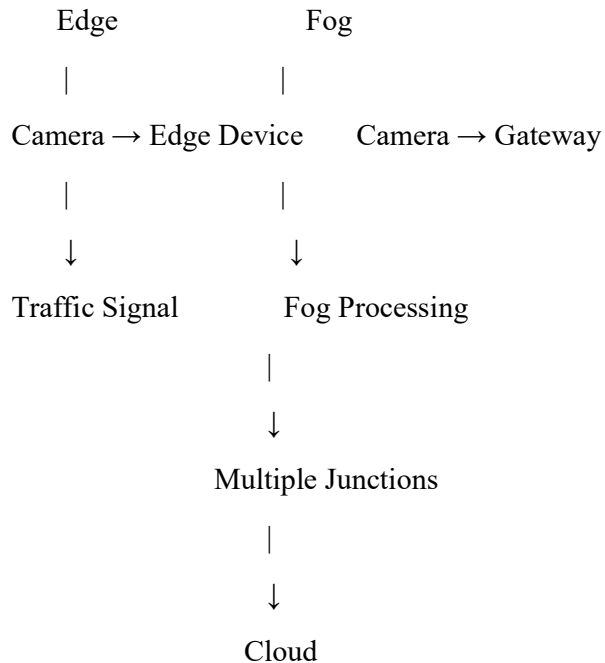
Parameter	Edge Computing	Fog Computing
Processing location	Directly at device/edge	Local network/fog node
Latency	Very low	Low
Network usage	Very low	Low
Suitable for	Immediate decisions	Regional coordination
Example	Traffic signal control	City traffic management

Procedure:

1. Identify traffic cameras and sensors as data sources.
2. Design an edge architecture where each traffic junction processes data locally.
3. Design a fog architecture where multiple junctions send data to a nearby fog node.
4. Compare the distance between sensors and processing systems.
5. Analyze latency for emergency traffic decisions.
6. Compare bandwidth consumption.
7. Identify suitable deployment locations.
8. Determine which architecture is better for different traffic-management requirements.

Expected Output

SMART TRAFFIC SYSTEM



Analysis

Edge computing provides the lowest latency because processing occurs directly at or near the traffic device. Fog computing provides a wider local processing layer and is better when information from multiple traffic junctions must be coordinated.

Conclusion

For **instant traffic signal decisions**, edge computing is preferable due to extremely low latency. For **coordination across multiple traffic junctions**, fog computing is more suitable because it provides centralized local processing without sending every piece of data to the cloud.

EXPERIMENT 5: Cloud Application Deployment Workflow

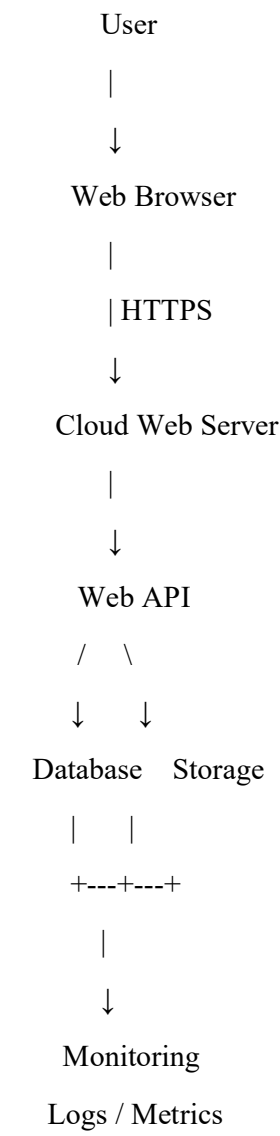
Title:

Demonstration of a Cloud Application Deployment and Monitoring Workflow

Objective

To demonstrate a complete cloud application workflow involving **client access, browser** interaction, web API communication, storage access, and application monitoring.

Architecture Sketch



Tools / Platforms

Possible tools include:

- Cloud VM or App Service
- Web browser
- REST API
- Cloud database
- Object storage
- Cloud monitoring service
- Git/GitHub for source-code management

Procedure

1. Develop a simple cloud application.
2. Create a cloud environment for deployment.
3. Upload or push the application source code.
4. Configure the application server.
5. Deploy the application.
6. Open the application URL in a browser.
7. Send a request from the browser to the web API.
8. Let the API access the required database or storage service.
9. Return the response to the browser.
10. Enable monitoring for application health, CPU usage, requests, errors, and logs.
11. Generate test requests.
12. Observe the application logs and monitoring metrics.

Expected Output

User



Browser



HTTPS Request



Cloud Application



Web API



Database / Object Storage



API Response



Browser

Example:

Client Request

GET /api/files



Cloud API



Storage Access



HTTP 200 OK



File Information Displayed

Monitoring Output

Application Status : Running

API Requests : Successful

Storage Access : Successful

Errors : 0

Application Health : Healthy

Analysis

The experiment demonstrates how different cloud components work together. The browser acts as the client, the web server handles application requests, the API provides backend

Conclusion

The experiment successfully demonstrated a complete cloud application deployment workflow, from client access and API communication to storage interaction and monitoring.